cNEAR

Security Assessment

August 14, 2026

Prepared for:
David Norris
Phillip Suarez
cAssets Management Ltd.

Prepared by: **Samuel Moelius and Marcelo Morales**

Table of Contents

Project Summary

Contact Information

The following project manager was associated with this project:

Kimberly Espinoza, Project Manager
kimberly.espinoza@trailofbits.com

The following engineering director was associated with this project:

Benjamin Samuels, Engineering Director, Blockchain
benjamin.samuels@trailofbits.com

The following consultants were associated with this project:

Samuel Moelius, Consultant
samuel.moelius@trailofbits.com

Marcelo Morales, Consultant
marcelo.morales@trailofbits.com

Project Timeline

The significant events and milestones of the project are listed below.

Date	Event
July 16, 2026	Pre-project kickoff call
July 27, 2026	Delivery of report draft
July 27, 2026	Report readout meeting
August 14, 2026	Delivery of final comprehensive report

Executive Summary

Engagement Overview

cAssets Management Ltd. engaged Trail of Bits to review the security of the cNEAR token contracts.

cNEAR is built on top of the standard NEP-141 fungible token standard reference implementation. cNEAR adds functionality that NEP-141 does not provide. For example, an account with the `Owner` role can pause the cNEAR contract. Additionally, an `Owner` account can "force transfer" funds away from another account. Either of these operations might be performed for regulatory reasons.

A team of one consultant conducted the review from July 20 to July 24, 2026, for a total of one engineer-week of effort. Our testing efforts focused on finding flaws in the minting and transfer mechanisms. With full access to source code and documentation, we performed static and dynamic testing of the codebase, using automated and manual processes.

Observations and Impact

The project relies heavily on the deployment being correct, as this determines who can invoke privileged methods on the controller and the cNEAR token. The deployment script is written in Bash, and we identified several issues with it (TOB-CNEAR-2, TOB-CNEAR-3, TOB-CNEAR-4, TOB-CNEAR-5, TOB-CNEAR-6). Additionally, there is no evidence that `shellcheck` is run in CI.

We found several problems related to ownership of the cNEAR contract. First, the contract uses access-control-based ownership, though it seems to require only a single owner (TOB-CNEAR-9). Bugs in the implementation allow previous owners to retain their roles (TOB-CNEAR-13). Beyond causing bugs, the access-control mechanism introduces complexity that complicates the code and inhibits its maintainability.

There are bugs in the cNEAR contract's upgrade mechanism. First, the controller does not call `add_deployment_info`, which is necessary for both `upgrade` and `unrestricted_upgrade` to succeed (TOB-CNEAR-5). Second, a controller-mediated upgrade deploys malformed Wasm. Consequently, the upgrade itself can succeed, but the deployed Wasm is nonoperational (TOB-CNEAR-7).

The implementation of the new pause and force transfer functionalities seems largely correct. However, the pausing mechanism does not emit events (TOB-CNEAR-11); the pause functionality does not cover all of the operations it likely should (TOB-CNEAR-14); and, finally, the interaction between these two mechanisms should be better documented (TOB-CNEAR-12).

Recommendations

Based on the findings identified during the security review, Trail of Bits recommends that cAssets Management take the following steps before deploying the cNEAR contracts:

- **Remediate the findings disclosed in this report.** These findings should be addressed through direct fixes or broader refactoring efforts.
- **Fix the bugs in the deployment script and test it thoroughly.** Address each of the findings in this report concerning the deployment script. Regularly run `shellcheck` over the deployment script (including in CI) to identify potential problems with it.
- **Simplify the contract's ownership model and test it thoroughly.** Such simplification should likely include a refactor of cNEAR to use the `near-plugins`' `Ownable` mechanism, rather than an access control list. Finally, consider diagramming the components in a way that includes the owners. Such a diagram will aid in comprehension and future development.
- **Fix cNEAR's upgrade mechanism.** Even if the contract could be fixed through other means (e.g., redeployment), a working upgrade mechanism will reduce friction and increase user confidence. Test the mechanism thoroughly once it is fixed. The tests should include verification that an upgraded contract is functional.
- **Improve the implementation of the pause and force transfer functionalities.** Ensure that the new functionalities emit sufficient events to allow off-chain monitoring. Ensure that the pause functionality covers all necessary operations, including, for example, `storage_unregister`. Finally, ensure that the interaction between the new mechanisms is sufficiently documented.

Finding Severities and Categories

The following tables provide the number of findings by severity and category.

EXPOSURE ANALYSIS

Severity	*Count*
High	2
Medium	1
Low	4
Informational	7
Undetermined	0

CATEGORY BREAKDOWN

Category	*Count*
Access Controls	3
Auditing and Logging	2
Configuration	4
Data Validation	1
Denial of Service	1
Error Reporting	2
Patching	1

Project Goals

The engagement was scoped to provide a security assessment of the cAssets Management cNEAR contracts. Specifically, we sought to answer the following non-exhaustive list of questions:

- Can an unauthorized user mint tokens?
- Can an unauthorized user perform fake transfers?
- Is there sufficient guidance for using the cNEAR token in a liquidity pool?
- Can an unauthorized user pause the contract?

Project Targets

The engagement involved reviewing and testing the following target.

cNEAR

Repository	https://github.com/sig-net/cNEAR
Version	`c1944263327d8b4cb0b640824b33f37f78d7108d`
Type	Rust
Platform	NEAR

Project Coverage

This section provides an overview of the analysis coverage of the review, as determined by our high-level engagement goals. Our approaches included the following:

- **Build and test verification:** We verified that the target codebase builds and that the test suite passes.
- **Test coverage review:** We computed coverage of the tests using `cargo-llvm-cov`. Per the stated scope, we did not review the tests for bugs. However, when production code seemed to exhibit bugs, we did try to determine why the tests did not catch them.
- **Dependencies review:** We ran `cargo-audit` to determine whether the cNEAR contracts rely on vulnerable dependencies.
- **Static analysis:** We ran Clippy with the pedantic lints enabled, and Dylint with the general and supplementary lints enabled, and reviewed the results. We verified that all results produced were innocuous.
- **AI-assisted manual code review:** We used Claude (Opus 4.8 and Sonnet 3.5) and Codex (GPT-5.6 Sol) to summarize the codebase, its architecture, its primary components, and the ways in which those components interact. This effort identified the most security-relevant parts of the codebase and potential bugs therein. We manually reviewed and triaged each lead.

Coverage Limitations

Because of the time-boxed nature of testing work, it is common to encounter coverage limitations. The following list outlines the coverage limitations of the engagement and indicates system elements that may warrant further review:

- **We did not have sufficient time for this engagement.** While the amount of Rust code was small, the cNEAR contract interacts with other contracts (e.g., the Aurora controller and the standard NEP-141 fungible token), and it was necessary to understand those contracts as well. Additionally, the deployment script plays a central role in the security of the system, as it determines the initial roles assigned to the deployed contracts. Hence, we determined it necessary to review the deployment script. Finally, the `justfile` affects the security of the system as well. This report includes one finding related to the `justfile` (TOB-CNEAR-8), but we suspect there may be more.
- **The tests were not reviewed, as they were explicitly declared out of scope.** If NEAR performs a subsequent review of the code, we recommend including the tests.

Codebase Maturity Evaluation

Trail of Bits uses a traffic-light protocol to provide each client with a clear understanding of the areas in which its codebase is mature, immature, or underdeveloped. Deficiencies identified here often stem from root causes within the software development life cycle that should be addressed through standardization measures (e.g., the use of common libraries, functions, or frameworks) or training and awareness programs.

Category	Summary	Result
Arithmetic	The production contract performs little custom arithmetic and delegates balance, supply, refund, and storage accounting to standard NEAR components. Rust's checked debug behavior, typed `u128` values, and tests for zero amounts, excessive balances, storage boundaries, refunds, and supply reduction provide useful assurance. The deployment script's initial token supply is incorrect; however, this is easily fixable.	**Satisfactory**
Auditing	The standard fungible-token paths emit mint and transfer events, and the resolver and storage-unregistration paths log exceptional balance changes. However, cNEAR-specific pause, freeze, unfreeze, ownership, and price changes do not emit events. Finally, there are no event descriptions, and the repository lacks monitoring, alerting, and incident-response guidance.	**Moderate**
Authentication / Access Controls	Privileged cNEAR methods use role guards. However, the custom `owner_id` state and generic `Role::Owner` ACL can diverge, and ownership transfers leave the previous owner with ACL administrative authority. The repository describes controller and DAO mediation but does not pin the controller implementation or use two-step ownership transfer.	**Weak**
Complexity Management	The production contract is small, uses standard traits for token and storage behavior, and keeps most custom functions short and direct. Strong typing, library	**Moderate**

	delegation, passing Clippy checks, and shallow control flow make the code reviewable. However, complexity increases because ownership is represented in both custom state and a generic ACL. Additionally, the project relies on outdated, vulnerable dependencies.	
Cryptography and Key Management	The contract does not implement custom cryptography and relies on NEAR account identifiers, access keys, standard hashing, and established SDK components.	**Not Applicable**
Decentralization	The documented deployment places the token under an Aurora controller initialized with a DAO account, which is a positive separation between the token and its initial deployer. Nevertheless, effective ownership can upgrade code immediately, move user balances forcibly, pause transfers, freeze accounts, and change the published price. The repository provides no timelock. Retained account access keys can bypass contract-level governance.	**Weak**
Documentation	The README provides useful build, test, deployment, method, and controller-integration instructions, and the source documents the intended meaning of the published price. However, the repository lacks an architecture diagram, trust model, role descriptions, and a privilege matrix. Several statements do not match implementation or deployment behavior, including claims about all-transfer pausing and caller-funded storage.	**Satisfactory**
Low-Level Manipulation	The codebase contains no unsafe Rust, inline assembly, or custom bitwise implementation. Its upgrade entry point nevertheless performs a security-critical low-level operation by reading the complete raw call input and deploying those bytes as contract code without decoding or validating the expected controller argument envelope. The operation has no repository test that proves the installed bytes are executable.	**Moderate**
Testing and Verification	The repository has a substantial baseline of unit and sandbox integration tests, and all executable non-controller tests pass. CI runs builds and tests on	**Moderate**

	Linux and macOS. Formatting and Clippy pass when invoked manually. However, the audited tests omit several cNEAR-specific authorization and interaction properties, and the controller upgrade test accepts initiation success without calling the upgraded token afterward. The repository would benefit from a coverage gate, fuzzing, property tests, invariant tests, mutation testing, or formal verification.	
Transaction Ordering	The fact the NEAR is sharded adds complication to how transfers are performed in NEAR. This, in turn, complicates transaction ordering. Nonetheless, we could find no transaction ordering risks specific to the cNEAR contracts.	**Satisfactory**

Summary of Findings

The table below summarizes the findings of the review, including details on type and severity.

ID	Title	Type	Severity
1	Locked dependencies contain known security vulnerabilities	Patching	**Informational**
2	Deployment retains access keys that bypass contract-governed administration	Configuration	**Informational**
3	Default deployment creates only one-billionth of one cNEAR	Configuration	**Informational**
4	Account-existence checks treat RPC errors as existing accounts	Error Reporting	**Low**
5	Deployment omits controller registration required for cNEAR upgrades	Configuration	**Low**
6	Ownership verification failures do not fail deployment	Error Reporting	**Low**
7	Controller-mediated upgrades deploy malformed contract code	Data Validation	**High**
8	Unpinned controller source can change privileged deployment code	Configuration	**Informational**
9	owner_get can disagree with effective ownership	Auditing and Logging	**Informational**
10	Freeze-list growth uses contract-funded storage contrary to documentation	Denial of Service	**Informational**
11	Pause, freeze, and price changes do not emit dedicated events	Auditing and Logging	**Low**

12	Force transfers bypass the pause and freeze controls without documentation	Access Controls	**Informational**
13	Ownership transfer retains the previous owner's administrative permissions	Access Controls	**High**
14	Frozen or paused accounts can burn their balance via storage_unregister, defeating owner recovery	Access Controls	**Medium**

Detailed Findings

1. Locked dependencies contain known security vulnerabilities

Severity: **Informational**	Difficulty: **High**
Type: Patching	Finding ID: TOB-CNEAR-1
Target: `Cargo.lock`	

Description
The locked Rust dependency graph contains 15 vulnerabilities recorded in the RustSec advisory database. These vulnerabilities include memory-safety defects, denial-of-service conditions, certificate-validation errors, noncanonical serialization, and unsafe archive extraction behavior.

The affected locked packages and available remediations are as follows:

Package	Locked version	Advisory	Patched version
`bytes`	1.8.0	RUSTSEC-2026-0007	1.11.1 or later
`crossbeam-epoch`	0.9.18	RUSTSEC-2026-0204	0.9.20 or later
`hashbrown`	0.15.0	RUSTSEC-2024-0402	0.15.1 or later
`idna`	0.5.0	RUSTSEC-2024-0421	1.0.0 or later
`openssl`	0.10.68	RUSTSEC-2025-0004	0.10.70 or later
`openssl`	0.10.68	RUSTSEC-2025-0022	0.10.72 or later
`ring`	0.17.8	RUSTSEC-2025-0009	0.17.12 or later
`rustls`	0.23.15	RUSTSEC-2024-0399	0.23.18 or later

Package	Locked version	Advisory	Patched version
`rustls-webpki`	0.102.8	RUSTSEC-2026-0049	0.103.10 or later
`rustls-webpki`	0.102.8	RUSTSEC-2026-0098	0.103.12 or later
`rustls-webpki`	0.102.8	RUSTSEC-2026-0099	0.103.12 or later
`rustls-webpki`	0.102.8	RUSTSEC-2026-0104	0.103.13 or later
`tar`	0.4.42	RUSTSEC-2026-0067	0.4.45 or later
`tar`	0.4.42	RUSTSEC-2026-0068	0.4.45 or later
`time`	0.3.36	RUSTSEC-2026-0009	0.3.47 or later

During the audit, `cargo audit` reproduced all 15 vulnerabilities. Running `cargo update` in an isolated copy of the audited repository and rescanning its updated lockfile eliminated all 15 vulnerability reports without changing the manifest. This result indicates that compatible patched dependency versions are already available.

Exploit Scenario

Mallory discovers a path to an exploitable bug in a vulnerable dependency. Mallory exploits the bug to cause financial harm to cNEAR token users.

Recommendations

Short term, run `cargo update`, review the resulting lockfile changes, execute the complete unit and integration test suite, rebuild the release Wasm, and commit the updated lockfile after confirming that `cargo audit` reports no vulnerabilities. This prevents the issue because dependency resolution selects patched versions that no longer contain the reported defects.

Long term, run `cargo audit` in CI with a policy that fails on vulnerabilities, review allowed warnings separately, and automate dependency update proposals so maintainers can test and merge patched versions promptly. This prevents recurrence because newly disclosed vulnerabilities become visible and release-blocking before vulnerable versions remain in production or development workflows.

2. Deployment retains access keys that bypass contract-governed administration

Severity: **Informational**	Difficulty: **High**
Type: Configuration	Finding ID: TOB-CNEAR-2
Target: `scripts/deploy.sh`	

Description

The production deployment flow does not verify or remove access keys from the cNEAR token account after transferring ownership to the controller (i.e., the deployment script contains no commands of the form 2.1). A holder of a retained full-access key can replace the token contract or delete its account without satisfying cNEAR's `Owner` check or the controller's role-based upgrade policy.

```
near delete-key <account-id> <public-key> \
  --accountId <account-id> \
  --networkId mainnet
```

Figure 2.1: Format of a command to remove an access key

NEP-141 recommends that a deployed fungible-token contract have no access keys to prevent modification or deletion outside the contract's authorization model. The cNEAR source repeats this guidance, but the deployment flow does not enforce it.

The production branch finishes by recommending functional checks, but it neither identifies access-key removal as a finalization step nor verifies that the token account has reached the intended keyless state. Retained controller-account keys present a related defense-in-depth concern because they permit protocol-level replacement or deletion of the controller. However, the NEP-141 recommendation specifically applies to the token account.

```
else
    echo ""
    echo "Next steps:"
    echo "  1. Add release info to controller"
    echo "  2. Test pause/freeze via controller"
    echo "  3. Test upgrades via controller"
fi
```

Figure 2.2: Production next steps that omit access-key verification and removal (cNEAR/scripts/deploy.sh#L375–L381)

Exploit Scenario

An attacker compromises a full-access private key retained on the cNEAR token account after deployment. The attacker submits a `DeployContract` action that replaces cNEAR with malicious code, bypassing both cNEAR's `Owner` authorization and the controller's DAO and updater policies. The replacement code can alter token balances or disable token operations; alternatively, the attacker can submit a `DeleteAccount` action and make the token unavailable.

Recommendations

Short term, add explicit production-finalization instructions and tooling that list the access keys on the token and controller accounts after the release, pause, freeze, and upgrade checks complete. Require the operator to confirm those checks before removing every token-account key and every controller-account key not required by the documented governance or emergency-recovery model (if such a document exists). Finally, verify the resulting on-chain key sets.

Long term, define an auditable deployment lifecycle that separates deployment and operational validation. Document the approved key configuration for each account, protect any deliberately retained emergency keys with controls commensurate with their ability to replace or delete contracts. Monitor the accounts for key changes. Finally, before each production deployment, test the finalization procedure on testnet.

References

- NEP-141 fungible-token standard
- NEAR access-key permissions

3. Default deployment creates only one-billionth of one cNEAR

Severity: **Informational**	Difficulty: **Low**
Type: Configuration	Finding ID: TOB-CNEAR-3
Target: `scripts/deploy.sh`, `src/lib.rs`	

Description

The deployment script combines 24 decimal places with a default total supply of `10^15` atomic units. A deployment that uses these defaults creates a total display supply of `10^15 / 10^24`, or `0.000000001` cNEAR, and the contract provides no method to mint the missing supply after initialization.

Interactive deployment presents both values as defaults. An operator can replace the total supply, but accepting the prompt values produces the undersized supply.

```
read -p "Token symbol [cNEAR]: " TOKEN_SYMBOL
TOKEN_SYMBOL=${TOKEN_SYMBOL:-"cNEAR"}

read -p "Token decimals [24]: " TOKEN_DECIMALS
TOKEN_DECIMALS=${TOKEN_DECIMALS:-24}

read -p "Total supply [1000000000000000]: " TOTAL_SUPPLY
TOTAL_SUPPLY=${TOTAL_SUPPLY:-"1000000000000000"}

read -p "Initial price in yoctoNEAR [1000000000000000000000000 (1 NEAR)]: "
INITIAL_PRICE
INITIAL_PRICE=${INITIAL_PRICE:-"1000000000000000000000000"}
```

Figure 3.1: Interactive deployment defaults to 24 decimals and `10^15` atomic units. (cNEAR/scripts/deploy.sh#L194–L204)

Noninteractive and test-mode deployments do not offer an override through the script's command-line interface and assign the same values unconditionally.

```
TOKEN_NAME="Controlled NEAR"
TOKEN_SYMBOL="cNEAR"
TOKEN_DECIMALS=24
TOTAL_SUPPLY="1000000000000000"
INITIAL_PRICE="1000000000000000000000000"
INITIAL_BALANCE=10
```

Figure 3.2: Noninteractive and test-mode deployment uses the undersized supply unconditionally. (cNEAR/scripts/deploy.sh#L212–L217)

The script passes the values directly to token initialization without converting a whole-token supply into atomic units.

```
DEPLOY_TOKEN_CMD="$NEAR_CMD deploy $TOKEN_ID $TOKEN_WASM --initFunction new
--initArgs
'{\"owner_id\":\"$SIGNER_ID\",\"total_supply\":\"$TOTAL_SUPPLY\",\"metadata\":{\"spe
c\":\"ft-1.0.0\",\"name\":\"$TOKEN_NAME\",\"symbol\":\"$TOKEN_SYMBOL\",\"decimals\":
$TOKEN_DECIMALS},\"latest_price\":\"$INITIAL_PRICE\"}' --networkId $NETWORK"
```

Figure 3.3: Deployment forwards the default supply and decimals directly to cNEAR. (cNEAR/scripts/deploy.sh#L318–L318)

Initialization then credits exactly `total_supply` atomic units to the initial owner and emits the corresponding mint event.

```
this.token.internal_register_account(&owner_id);
this.token.internal_deposit(&owner_id, total_supply.into());

near_contract_standards::fungible_token::events::FtMint {
    owner_id: &owner_id,
    amount: total_supply,
    memo: Some("new tokens are minted"),
}
.emit();
```

Figure 3.4: Initialization mints exactly the supplied atomic-unit amount. (cNEAR/src/lib.rs#L87–L95)

If cNEAR's intended initial supply equals NEAR's initial supply of one billion tokens, the corresponding 24-decimal value is `10^9 × 10^24`, or `10^33`, atomic units.

If this interpretation reflects the intended token economics, the default should therefore be `10^33` atomic units rather than `10^15`. Because cNEAR mints its supply only during initialization, correcting an already-deployed instance requires replacing or migrating the contract rather than invoking an administrative mint operation.

Exploit Scenario

An operator follows the documented noninteractive deployment flow and supplies a signer without overriding token parameters. The script deploys a 24-decimal token with `10^15` atomic units, transfers administrative ownership to the controller, and reports success. When the issuer attempts to distribute whole cNEAR amounts, the entire supply equals only `0.000000001` cNEAR, and no post-initialization mint method exists to create the intended inventory. If cNEAR's intended initial supply equals NEAR's one-billion-token initial supply, the deployment is short by a factor of `10^18`, and the operator must replace or migrate it before issuance can proceed.

Recommendations

Short term, document the intended supply in both whole cNEAR and atomic units. If cNEAR's initial supply should equal NEAR's one-billion-token initial supply, set the 24-decimal atomic-unit default to `10^33`. Update the script to derive the atomic-unit supply with checked arithmetic, reject unrepresentable or implausibly small values, and require confirmation of both representations. This prevents the issue because deployment cannot silently use an undersized supply.

Long term, define token economics in a specification that states the supply baseline, decimals, atomic-unit representation, and maximum representable value. Add tests that run every deployment mode, decode the generated initialization arguments, and assert the intended decimals, atomic supply, human-readable supply, and initial owner balance.

4. Account-existence checks treat RPC errors as existing accounts

Severity: **Low**	Difficulty: **Medium**
Type: Error Reporting	Finding ID: TOB-CNEAR-4
Target: `scripts/deploy.sh`	

Description

The deployment script determines whether the controller and token accounts exist by searching the NEAR CLI's combined output for the substring `Account`. Errors containing terms such as `ViewAccount` therefore cause the script to classify an unsuccessful query as proof that the account exists, skip account creation, and continue from an invalid assumption.

The controller check redirects standard error into standard output and disregards the NEAR command's status because `grep` determines the pipeline result. If `grep` finds `Account` anywhere in the output, the surrounding expression emits `true`; otherwise, it emits `false`. Both branches terminate successfully, so `set -e` does not detect the failed query.

```
# Check controller account
if [[ "$DRY_RUN" == "false" ]]; then
    CONTROLLER_EXISTS=$($NEAR_CMD state $CONTROLLER_ID --networkId $NETWORK 2>&1 |
grep -q "Account" && echo "true" || echo "false")
else
    CONTROLLER_EXISTS="unknown"
fi
```

Figure 4.1: Controller existence check treats any output containing `Account` as success. (cNEAR/scripts/deploy.sh#L273–L278)

The script repeats the same logic for the token account.

```
# Check token account
if [[ "$DRY_RUN" == "false" ]]; then
    TOKEN_EXISTS=$($NEAR_CMD state $TOKEN_ID --networkId $NETWORK 2>&1 | grep -q
"Account" && echo "true" || echo "false")
else
    TOKEN_EXISTS="unknown"
fi
```

Figure 4.2: Token existence check repeats the output-based classification. (cNEAR/scripts/deploy.sh#L291–L296)

An unsuccessful query from NEAR CLI 0.27.0 can include the following:

```
Failed to fetch query ViewAccount
```

The current check would classify that RPC failure as `true`. The subsequent deployment command will generally fail because the account does not exist or cannot be queried, but the misleading classification obscures the original cause and can leave a partially completed deployment when one account check succeeds and the other is misclassified.

Exploit Scenario

An attacker disrupts access to the configured RPC endpoint while an operator runs the deployment script. The account query fails with an error containing `ViewAccount`, but the script reports that the account exists and skips its creation. A later deployment step fails after another account may already have been created, delaying deployment and requiring the operator to diagnose and clean up the partial state.

Recommendations

Short term, preserve and evaluate the NEAR CLI's exit status before inspecting its response. Treat a successful query as evidence that the account exists, handle the CLI's explicit account-not-found response as evidence that creation is required, and terminate on RPC, network, parsing, and configuration errors.

Long term, consume a structured RPC or CLI response with explicit success and error variants instead of matching human-readable output. Add deployment-script tests for existing accounts, absent accounts, RPC failures, malformed responses, and authentication failures, and assert that the script never maps an indeterminate result to either existence or nonexistence.

5. Deployment omits controller registration required for cNEAR upgrades

Severity: **Low**	Difficulty: **Low**
Type: Configuration	Finding ID: TOB-CNEAR-5
Target: `scripts/deploy.sh`, `README.md`	

Description

The deployment workflow transfers ownership of cNEAR to the Aurora controller without registering the token in the controller's deployment registry. As a result, the documented controller-mediated upgrade procedure will reject every cNEAR upgrade until an operator manually creates the missing deployment record.

The controller requires a deployment record before it constructs an upgrade promise. Both `upgrade` and `unrestricted_upgrade` reach this lookup and panic when the requested contract is absent.

```
let mut deployment_info =
    self.deployments
        .get(&contract_id)
        .cloned()
        .unwrap_or_else(|| {
            panic!("contract with account id: {contract_id} hasn't been deployed")
        });
```

Figure 5.1: Controller upgrade logic requires an existing deployment record. (aurora-controller-factory/contract/src/lib.rs#L532–L538)

The deployment script deploys the controller and token separately, then transfers cNEAR ownership to the controller. It does not call `add_deployment_info`.

```
# Step 1: Deploy controller
echo -e "${GREEN}=== Step 1: Deploy Controller ===${NC}"
DEPLOY_CONTROLLER_CMD="$NEAR_CMD deploy $CONTROLLER_ID $CONTROLLER_WASM
--initFunction new --initArgs '{\"dao\":\"$SIGNER_ID\"}' --networkId $NETWORK"

run_cmd "$DEPLOY_CONTROLLER_CMD"

# Step 2: Deploy token with signer as initial owner
echo -e "${GREEN}=== Step 2: Deploy Token with Signer as Initial Owner ===${NC}"
DEPLOY_TOKEN_CMD="$NEAR_CMD deploy $TOKEN_ID $TOKEN_WASM --initFunction new
--initArgs
'{\"owner_id\":\"$SIGNER_ID\",\"total_supply\":\"$TOTAL_SUPPLY\",\"metadata\":{\"spe
c\":\"ft-1.0.0\",\"name\":\"$TOKEN_NAME\",\"symbol\":\"$TOKEN_SYMBOL\",\"decimals\":
```

```
$TOKEN_DECIMALS},\"latest_price\":\"$INITIAL_PRICE\"}' --networkId $NETWORK"

run_cmd "$DEPLOY_TOKEN_CMD"

# Step 3: Transfer token ownership to controller
echo -e "${GREEN}=== Step 3: Transfer Token Ownership to Controller ===${NC}"
TRANSFER_OWNERSHIP_CMD="$NEAR_CMD call $TOKEN_ID owner_set
'{\"new_owner\":\"$CONTROLLER_ID\"}' --accountId $SIGNER_ID --networkId $NETWORK"

run_cmd "$TRANSFER_OWNERSHIP_CMD"
```

Figure 5.2: Deployment transfers cNEAR ownership without registering the deployment. (cNEAR/scripts/deploy.sh#L310–L326)

The documented upgrade procedure registers a release and uploads its code before calling `upgrade`, but it also omits `add_deployment_info`. Therefore, following the documentation does not repair the missing registry entry.

```
# Upgrade via controller
# 1. Add release info
near call <controller-id> add_release_info '{"hash": "<wasm-sha256>", "version":
"1.1.0", "is_latest": true}' --accountId <dao-account> --amount
0.00000000000000000000001

# 2. Upload wasm blob
near call <controller-id> add_release_blob --accountId <dao-account> --amount
0.00000000000000000000001 < token.wasm

# 3. Upgrade token
near call <controller-id> upgrade '{"contract_id": "<token-id>", "hash":
"<wasm-sha256>"}' --accountId <dao-account> --amount 0.00000000000000000000001
```

Figure 5.3: Documented upgrade procedure omits deployment registration. (cNEAR/README.md#L193–L201)

An administrator can repair the configuration because the DAO role can call `add_deployment_info`. However, discovering and correcting the omission during an emergency can delay deployment of a security update, and an incorrectly constructed record can disrupt version and downgrade tracking. Registration is necessary for controller-mediated upgrades, but it does not resolve the separately identified incompatibility between the controller's serialized upgrade arguments and cNEAR's raw-Wasm upgrade input.

A test that reproduces this failure appears in appendix C.

Exploit Scenario

An attacker begins exploiting a newly disclosed flaw in cNEAR. The DAO prepares a fixed release, uploads its code to the controller, and invokes `upgrade` according to the documented procedure. The controller finds no deployment record for cNEAR and rejects

the call, extending the period during which the attacker can exploit the vulnerable version while administrators diagnose and repair the controller configuration.

Recommendations

Short term, register the initial cNEAR release and its deployment information with the controller as part of the deployment finalization process. Query `get_deployment` and verify its hash, version, and contract ID before reporting deployment success or transferring operational responsibility.

Long term, maintain one tested deployment workflow that atomically establishes or verifies every controller invariant required for upgrades and downgrades. Add an integration test that follows the documented production commands from an empty controller state and confirms that the resulting deployment can enter the complete upgrade path.

6. Ownership verification failures do not fail deployment

Severity: **Low**	Difficulty: **Low**
Type: Error Reporting	Finding ID: TOB-CNEAR-6
Target: `scripts/deploy.sh`	

Description

The deployment script reports success even when it cannot verify that the controller owns cNEAR. A failed RPC request, unexpected CLI output, or ownership mismatch therefore produces contradictory output and a successful process exit, which can cause operators and automation to accept an unverified deployment.

The script suppresses errors from the ownership query, parses its human-readable output, and prints an error when the result does not match the expected controller. The failure branch does not exit or otherwise mark the deployment as unsuccessful.

```
# Step 4: Verify ownership
echo -e "${GREEN}=== Step 4: Verify Ownership ===${NC}"
VERIFY_CMD="$NEAR_CMD view $TOKEN_ID owner_get '{}' --networkId $NETWORK"

if [[ "$DRY_RUN" == "false" ]]; then
    echo -e "${BLUE}Verifying token owner...${NC}"
    OWNER_OUTPUT=$(eval "$VERIFY_CMD" 2>&1 || echo "")

    # Extract account ID - handle both quoted and unquoted output
    OWNER_RESULT=$(echo "$OWNER_OUTPUT" | grep -oE
'(controller\.[a-z0-9\-\.]+|"[^"]+")' | tr -d '"' | tail -1)

    if [[ -n "$OWNER_RESULT" && "$OWNER_RESULT" == "$CONTROLLER_ID" ]]; then
        echo -e "${GREEN}✓ Ownership verified: $OWNER_RESULT${NC}\n"
    else
        echo -e "${RED}✗ Ownership verification failed.${NC}"
        echo -e "Expected: ${BLUE}$CONTROLLER_ID${NC}"
        echo -e "Got:      ${BLUE}$OWNER_RESULT${NC}"
        echo -e "Raw output: $OWNER_OUTPUT\n"
    fi
else
    echo -e "${BLUE}Command:${NC} $VERIFY_CMD"
    echo -e "${YELLOW}[DRY RUN - not executed]${NC}\n"
fi
```

Figure 6.1: Ownership verification suppresses query errors and does not propagate mismatches. (cNEAR/scripts/deploy.sh#L328–L350)

This behavior weakens the verification step as a deployment control. In particular, an orchestration system that relies on the process exit status cannot distinguish a verified deployment from one for which the ownership query failed.

Exploit Scenario

An operator deploys cNEAR while the configured RPC endpoint is intermittent. The ownership transaction completes, but the subsequent view request fails, or a future change causes the ownership result to differ from the expected controller. The script prints the verification error, immediately reports that deployment is complete, and exits successfully; downstream automation proceeds with operational finalization despite lacking evidence that the intended authorization state was established.

Recommendations

Short term, preserve the ownership query's exit status and terminate with a nonzero status if the query fails, its output cannot be parsed, or the returned owner differs from the controller. Print the completion summary only after successful verification, and display the value returned by the contract rather than the expected value.

Long term, use structured CLI output or a direct RPC response instead of parsing human-readable console text. Add deployment-script tests that inject RPC errors, malformed output, empty output, and ownership mismatches, and assert that every case prevents the success summary and returns a failure status.

7. Controller-mediated upgrades deploy malformed contract code

Severity: **High**	Difficulty: **Low**
Type: Data Validation	Finding ID: TOB-CNEAR-7
Target: `src/lib.rs`	

Description

The cNEAR upgrade entrypoint treats its entire call input as raw Wasm, but the intended Aurora controller Borsh-serializes the Wasm together with an optional state-migration gas value. A controller-mediated upgrade therefore makes cNEAR deploy the serialized argument envelope instead of the Wasm contained within it. Subsequent calls fail during contract compilation, disabling transfers, administrative operations, and the contract's governed upgrade mechanism.

The cNEAR implementation declares an argument-free `upgrade` method, reads `env::input` directly, and passes every input byte to `deploy_contract`. This interface works when a caller supplies raw Wasm bytes without an argument encoding layer.

```
/// Upgrade contract code - owner only
#[access_control_any(roles(Role::Owner))]
pub fn upgrade(&self) -> Promise {
    let code = env::input().expect("no code provided").to_vec();
    Promise::new(env::current_account_id()).deploy_contract(code)
}
```

Figure 7.1: The upgrade method deploys its complete raw input. (cNEAR/src/lib.rs#L120–L125)

In contrast, the tested controller revision calls an external `upgrade` interface whose `code` and `state_migration_gas` parameters use Borsh serialization. Borsh encoding prefixes the Wasm with the length of the `Vec<u8>` and appends the encoded optional gas value. The resulting input does not begin with the Wasm magic bytes because it begins with the vector-length prefix.

```
fn upgrade_promise(
        contract_id: AccountId,
        args: UpgradeArgs,
        deployment_info: DeploymentInfo,
    ) -> Promise {
        ext_aurora::ext(contract_id.clone())
            .with_static_gas(
                args.state_migration_gas
```

```
                    .map_or(UPGRADE_GAS_NO_MIGRATION_GAS, |gas| {
                        UPGRADE_GAS.saturating_add(Gas::from_gas(gas))
                    }),
            )
            .with_unused_gas_weight(1)
            .with_attached_deposit(NearToken::from_yoctonear(1))
            .upgrade(args.code, args.state_migration_gas)
            .then(
                Self::ext(env::current_account_id())
                    .with_static_gas(ADD_DEPLOYMENT_GAS)
                    .with_unused_gas_weight(0)
                    .update_deployment_info(contract_id, deployment_info),
            )
    }
}

#[ext_contract(ext_aurora)]
pub trait ExtAurora {
    /// Requires 1yN attached for security purposes
    fn upgrade(
        &mut self,
        #[serializer(borsh)] code: Vec<u8>,
        #[serializer(borsh)] state_migration_gas: Option<u64>,
    );
}
```

Figure 7.2: The Aurora controller Borsh-serializes two upgrade arguments. (aurora-controller-factory/contract/src/lib.rs#L567–L599)

The self-contained NEAR sandbox reproduction in appendix C registers cNEAR with the controller, invokes `unrestricted_upgrade` with valid cNEAR Wasm, and confirms that the controller transaction reports success. Its final health check then calls `owner_get` and requires the call to fail with a `CompilationError`. This result confirms that controller success does not imply that the target account contains executable contract code. Because the malformed contract cannot execute `upgrade`, the controller cannot use the intended upgrade mechanism to repair the deployment.

Exploit Scenario

An attacker compromises an account authorized to initiate controller upgrades and selects an approved cNEAR release. The attacker invokes the controller's upgrade operation, which Borsh-serializes the valid Wasm and migration-gas argument before calling cNEAR. cNEAR deploys the serialized envelope as contract code, and every subsequent token call fails with a Wasm deserialization error. Users can no longer transfer tokens, and governance cannot repair the contract through its intended upgrade mechanism.

Recommendations

Short term, change cNEAR's `upgrade` interface to Borsh-decode the `code` and `state_migration_gas` arguments expected by the controller and pass only the decoded `code` bytes to `deploy_contract`. Reject nonempty migration-gas values until cNEAR

implements and tests a corresponding state-migration flow. Add an integration test that upgrades through the controller and then verifies token balances, ownership, pause state, frozen-account state, and another authorized upgrade. This prevents the issue because serialization metadata will no longer be interpreted as executable Wasm, and the test will verify the target contract rather than only the initiating controller transaction.

Long term, document the upgrade ABI shared by the controller and every controlled contract, and validate this ABI in CI whenever either component changes. Upgrade verification should inspect all receipt outcomes and call the upgraded contract before recording the deployment as successful. This prevents similar issues because incompatible controller and target revisions will fail before deployment instead of corrupting production contract code.

8. Unpinned controller source can change privileged deployment code

Severity: **Informational**	Difficulty: **High**
Type: Configuration	Finding ID: TOB-CNEAR-8
Target: `justfile`	

Description

The build process retrieves the Aurora controller without selecting an immutable commit, tag, or content hash. Builds performed at different times or in different workspaces can therefore compile different controller source code, including an upstream revision that has not been reviewed with cNEAR.

The setup recipe clones the repository's current default branch when the checkout directory does not exist. It does not check out or verify a specific revision. If the directory already exists, the recipe instead builds its existing contents without verifying their origin, revision, or cleanliness.

```
# Setup aurora-controller if not present
setup-controller:
    #!/usr/bin/env bash
    TARGET_DIR="${CARGO_TARGET_DIR:-.}"
    if [ ! -d "$TARGET_DIR/contracts/aurora-controller-factory" ]; then
        mkdir -p "$TARGET_DIR/contracts"
        git clone https://github.com/aurora-is-near/aurora-controller-factory.git
"$TARGET_DIR/contracts/aurora-controller-factory"
    fi

# Build aurora-controller using cargo make (puts wasm in res/)
build-controller: setup-controller
    #!/usr/bin/env bash
    cd contracts/aurora-controller-factory
    cargo make build
    # Copy wasms back to repo root's target/near dir
    cd ../..
    mkdir -p target/near
    cp contracts/aurora-controller-factory/res/aurora-controller-factory.wasm
target/near/ 2>/dev/null || true
```

Figure 8.1: The build clones and compiles the controller without selecting or verifying a revision. (cNEAR/justfile#L3–L20)

The deployment process uses the resulting Wasm and then transfers cNEAR ownership to the deployed controller. The controller consequently occupies a security-critical trust

boundary: its code mediates privileged operations over cNEAR, but the cNEAR repository does not identify the controller source revision from which that code must be built.

This design prevents reproducible review of the combined system. A passing integration test or successful deployment identifies neither the controller source revision tested nor whether production uses the same revision.

Exploit Scenario

An attacker compromises an upstream maintainer account and modifies the controller repository's default branch before a cNEAR release build. An operator runs the documented build in a fresh workspace, which clones and compiles the modified revision without displaying or validating an expected commit. The deployment script installs the malicious controller and transfers cNEAR ownership to it. The controller can then misuse its privileged relationship with cNEAR, while the cNEAR commit and release records do not reveal which controller source produced the deployed Wasm.

Recommendations

Short term, record an audited controller commit in the cNEAR repository, fetch and check out that exact commit during the build, reject dirty or unexpected controller worktrees, and verify the resulting Wasm against an approved digest before deployment. This prevents the issue because controller source and artifacts must match an explicitly reviewed, immutable identity.

Long term, make the controller a reproducible dependency through a commit-locked submodule, vendored source tree, or equivalent dependency mechanism. Publish the controller commit, toolchain, build command, and expected Wasm digest with each cNEAR release, and enforce those values in CI and deployment tooling. This prevents recurrence because review, testing, and production deployment remain bound to the same controller implementation.

9. owner_get can disagree with effective ownership

Severity: **Informational**	Difficulty: **High**
Type: Auditing and Logging	Finding ID: TOB-CNEAR-9
Target: `src/lib.rs`	

Description

cNEAR reports a singular `owner_id` through `owner_get` while independently authorizing every account that holds `Role::Owner`. An outgoing owner can grant this role to another account before transferring ownership, causing the additional account to retain access to all owner-restricted operations after the transfer while `owner_get` reports only the new owner. As a result, integrations cannot rely on `owner_get` to identify every account with effective ownership authority.

The contract defines `Role::Owner` through the generic access-control component and separately stores `owner_id`. The access-control component exposes role-management methods, including `acl_grant_role`, but the contract does not ensure that the `Owner` role has exactly one member or that its member equals `owner_id`.

```
#[derive(AccessControlRole, Clone, Copy)]
#[near(serializers = [json])]
enum Role {
    Owner,
}

#[derive(PanicOnDefault)]
#[access_control(role_type(Role))]
#[near(contract_state)]
pub struct Contract {
    token: FungibleToken,
    metadata: LazyOption<FungibleTokenMetadata>,
    frozen_accounts: LookupSet<AccountId>,
    paused: bool,
    owner_id: AccountId,
    latest_price: u128,
}
```

Figure 9.1: The contract maintains independent ACL and owner-state representations. (cNEAR/src/lib.rs#L37–L53)

The helper that establishes an owner makes the account a super-admin, an administrator of the `Owner` role, and a member of the `Owner` role. The owner can therefore use the generated ACL interface to grant `Role::Owner` to additional accounts.

During an ownership transfer, `owner_set` revokes the `Owner` role only from the account stored in `owner_id`. It does not identify or revoke any other `Owner`-role members. After the transfer, these additional members can invoke methods protected by `Role::Owner`, although `owner_get` returns only the new `owner_id`.

```
pub fn owner_get(&self) -> AccountId {
    self.owner_id.clone()
}

#[access_control_any(roles(Role::Owner))]
pub fn owner_set(&mut self, new_owner: AccountId) {
    let old_owner = self.owner_id.clone();
    self.owner_id = new_owner.clone();
    // Transfer role to new owner
    self.acl_revoke_role(Role::Owner.into(), old_owner);
    self.grant_owner_role(&new_owner);
}

fn grant_owner_role(&mut self, owner: &AccountId) {
    let mut acl = self.acl_get_or_init();
    acl.add_super_admin_unchecked(owner);
    acl.add_admin_unchecked(Role::Owner, owner);
    acl.grant_role_unchecked(Role::Owner, owner);
}
```

Figure 9.2: Ownership transfer updates only the reported owner and its corresponding role membership. (cNEAR/src/lib.rs#L156–L174)

Note that this finding is about data reporting. TOB-CNEAR-13 discusses the negative implications of additional `Owners` retaining their roles.

Exploit Scenario

Before transferring cNEAR to a successor or governance controller, a malicious owner calls `acl_grant_role` to grant `Role::Owner` to an accomplice. The owner then calls `owner_set`, which updates `owner_id`, revokes the outgoing owner's role, and grants ownership permissions to the successor. The accomplice retains `Role::Owner` and can subsequently upgrade the contract, force-transfer tokens, freeze accounts, pause transfers, change the published price, or perform another ownership transfer. Integrations that call `owner_get` continue to report only the legitimate successor and do not reveal the accomplice's authority.

Recommendations

Short term, prevent the generic ACL interface from granting or revoking `Role::Owner` outside `owner_set`. During every ownership transfer, assert that the outgoing owner is the sole `Owner`-role member, grant the required permissions to the new owner, revoke every ownership-related permission from the outgoing owner, and verify that exactly one `Owner`-role member remains and equals `owner_id`.

Long term, take the following steps:

- Use the `Ownable` component provided by the `near-plugins` dependency, which can supply singular ownership storage and `#[only(owner)]` authorization.
- Implement a two-step transfer in which the current owner proposes a pending owner and that account must explicitly accept ownership. Note that the standard `owner_set` interface transfers ownership immediately and does not enforce acceptance; do not expose that single-step path as an alternative to the proposal-and-acceptance flow.
- Account for ownership renunciation, controller compatibility, and storage migration, and add invariant tests confirming that `owner_get` always identifies the only account capable of invoking owner-restricted methods, that only the pending owner can accept a transfer, and that proposing an invalid or inaccessible account does not immediately relinquish control.

These steps prevent the issue because effective authority remains singular and cannot move until the intended recipient proves control of the destination account.

10. Freeze-list growth uses contract-funded storage contrary to documentation

Severity: **Informational**	Difficulty: **High**
Type: Denial of Service	Finding ID: TOB-CNEAR-10
Target: `src/lib.rs`	

Description

cNEAR funds every new freeze-list entry from the contract account even though its file-level documentation states that callers fund storage growth and receive refunds when storage decreases. A sufficiently large freeze list can lock the contract's available NEAR balance and cause a later emergency freeze to fail, while the documentation gives operators no indication that they must fund or monitor this administrative storage.

The file-level comment describes a uniform storage-accounting policy: the contract measures storage changes, requires an attached deposit for growth, refunds released storage, and returns excess deposits.

```
/*!
Fungible Token implementation with JSON serialization.
NOTES:
  - The maximum balance value is limited by U128 (2**128 - 1).
  - JSON calls should pass U128 as a base-10 string. E.g. "100".
  - The contract optimizes the inner trie structure by hashing account IDs. It will
prevent some
    abuse of deep tries. Shouldn't be an issue, once NEAR clients implement full
hashing of keys.
  - The contract tracks the change in storage before and after the call. If the
storage increases,
    the contract requires the caller of the contract to attach enough deposit to the
function call
    to cover the storage cost.
    This is done to prevent a denial of service attack on the contract by taking all
available storage.
    If the storage decreases, the contract will issue a refund for the cost of the
released storage.
    The unused tokens from the attached deposit are also refunded, so it's safe to
    attach more deposit than required.
```

Figure 10.1: The file-level documentation states that callers fund storage growth and receive storage refunds. (cNEAR/src/lib.rs#L1–L14)

The cNEAR-specific freeze methods do not implement that policy. `freeze_account` is not payable, does not measure storage growth, and inserts any supplied account ID into a

persistent `LookupSet`. Each previously absent account therefore increases storage usage and locks additional NEAR from the contract account. `unfreeze_account` removes the entry and releases the corresponding storage stake back to the contract, but it does not issue a refund to the caller or affected account.

```
#[access_control_any(roles(Role::Owner))]
pub fn freeze_account(&mut self, account_id: AccountId) {
    self.frozen_accounts.insert(account_id);
}

#[access_control_any(roles(Role::Owner))]
pub fn unfreeze_account(&mut self, account_id: AccountId) {
    self.frozen_accounts.remove(&account_id);
}
```

Figure 10.2: Freeze-list mutations neither charge nor refund their callers for storage. (cNEAR/src/lib.rs#L127–L135)

The `Owner`-role restriction prevents arbitrary users from filling the collection. However, an erroneous administrative batch, a compromised owner, or an unexpectedly large compliance list can still reduce the contract's available balance. Once the account lacks enough available NEAR to cover another entry, the runtime rejects further storage growth and rolls back the requested freeze.

Exploit Scenario

An attacker compromises an account authorized as `Owner` and calls `freeze_account` with many distinct account IDs. Each call adds a persistent entry without contributing a storage deposit, progressively locking cNEAR's available NEAR balance. When operators later attempt to freeze an account involved in an active incident, the call fails because the contract cannot cover the additional storage stake. Operators must fund the contract or identify and unfreeze another account before applying the emergency control.

Recommendations

Short term, document that cNEAR subsidizes freeze-list storage and ensure that the contract maintains a funded storage reserve. Track or bound the number of frozen accounts, reject additions before they consume the reserve, and alert operators when available storage capacity approaches the configured threshold. If the owner should fund each entry instead, make `freeze_account` payable, charge its measured storage increase, and define who receives the released storage stake after `unfreeze_account`.

Long term, define one explicit storage-funding policy for every cNEAR-specific variable-size data structure. Add tests that measure storage before and after each insertion and removal, verify the intended payer and beneficiary, and exercise behavior near the contract's minimum storage balance. Keep file-level and deployment documentation synchronized with this policy.

11. Pause, freeze, and price changes do not emit dedicated events

Severity: **Low**	Difficulty: **High**
Type: Auditing and Logging	Finding ID: TOB-CNEAR-11
Target: `src/lib.rs`	

Description

cNEAR changes its pause status, account-freeze status, and owner-published reference price without emitting dedicated events. An attacker who compromises the owner can therefore make security-relevant administrative changes without producing event records that identify the action, affected account, or new value. Monitoring and indexing systems must reconstruct these changes from transaction inputs or repeatedly query contract state, increasing the risk of delayed or inconsistent detection.

The `pause`, `pause_contract`, and `unpause` methods update the `paused` field but do not log the resulting state transition. Because `pause_contract` delegates to `pause`, both entry points have the same behavior.

```
#[access_control_any(roles(Role::Owner))]
pub fn pause(&mut self) {
    self.paused = true;
}

/// Alias for pause() - matches Aurora Controller expected method name
#[access_control_any(roles(Role::Owner))]
pub fn pause_contract(&mut self) {
    self.pause();
}

#[access_control_any(roles(Role::Owner))]
pub fn unpause(&mut self) {
    self.paused = false;
}

pub fn is_paused(&self) -> bool {
    self.paused
}
```

Figure 11.1: Pause state changes without a dedicated event. (cNEAR/src/lib.rs#L100–L118)

Similarly, `freeze_account`, `unfreeze_account`, and `set_latest_price` modify the operational state without emitting events that identify the action, actor, affected account, or new value.

```rust
#[access_control_any(roles(Role::Owner))]
pub fn freeze_account(&mut self, account_id: AccountId) {
    self.frozen_accounts.insert(account_id);
}

#[access_control_any(roles(Role::Owner))]
pub fn unfreeze_account(&mut self, account_id: AccountId) {
    self.frozen_accounts.remove(&account_id);
}

pub fn is_frozen(&self, account_id: AccountId) -> bool {
    self.frozen_accounts.contains(&account_id)
}

#[access_control_any(roles(Role::Owner))]
pub fn set_latest_price(&mut self, price: U128) {
    self.latest_price = price.into();
}
```

Figure 11.2: Freeze and price state changes without dedicated events (cNEAR/src/lib.rs#L127–L143)

The absence of freeze and unfreeze events is particularly relevant because the contract stores frozen accounts in a `LookupSet`. This collection supports membership checks but does not retain an iterable index of its elements.

```rust
#[derive(PanicOnDefault)]
#[access_control(role_type(Role))]
#[near(contract_state)]
pub struct Contract {
    token: FungibleToken,
    metadata: LazyOption<FungibleTokenMetadata>,
    frozen_accounts: LookupSet<AccountId>,
    paused: bool,
    owner_id: AccountId,
    latest_price: u128,
}
```

Figure 11.3: Frozen accounts are stored in a non-iterable collection. (cNEAR/src/lib.rs#L43–L53)

The public `is_frozen` method can answer whether a known account is frozen, but the contract exposes no method to enumerate every frozen account. An indexer that starts after deployment or misses part of the transaction history cannot recover the complete freeze list from a contract-state query. It must discover candidate accounts elsewhere and query them individually or replay every successful freeze and unfreeze call. Dedicated events would provide an explicit history from which off-chain systems can maintain the otherwise unavailable enumerable view.

The standard `near-contract-standards` NEP-141 implementation used by cNEAR illustrates the expected event-emission pattern for token state changes. Its `internal_transfer` method updates the sender and receiver balances and then emits a structured `FtTransfer` event containing the affected accounts, amount, and optional memo.

```
pub fn internal_transfer(
    &mut self,
    sender_id: &AccountId,
    receiver_id: &AccountId,
    amount: Balance,
    memo: Option<String>,
) {
    require!(sender_id != receiver_id, "Sender and receiver should be different");
    require!(amount > 0, "The amount should be a positive number");
    self.internal_withdraw(sender_id, amount);
    self.internal_deposit(receiver_id, amount);
    FtTransfer {
        old_owner_id: sender_id,
        new_owner_id: receiver_id,
        amount: U128(amount),
        memo: memo.as_deref(),
    }
    .emit();
}
```

Figure 11.4: The standard NEP-141 implementation emits a structured event after changing token balances.
(near-sdk-rs/near-contract-standards/src/fungible_token/core_impl.rs#L94–L112)

The missing events do not conceal the transactions or resulting state from the chain, but they require monitoring systems to parse calls or poll state rather than consume a stable event interface. The resulting impact is delayed detection and response rather than unauthorized state access.

Exploit Scenario

An attacker compromises an account authorized as `Owner` and unpauses cNEAR, unfreezes a monitored account, or publishes a new reference price. An off-chain monitor subscribed to contract events receives no event describing the administrative change and detects it only through transaction parsing or later state polling. Incident responders consequently continue operating with stale pause, freeze, or price information and respond to the compromised owner later than they would if each administrative transition produced a dedicated event.

Recommendations

Short term, define and emit dedicated structured events after every effective pause, unpause, freeze, unfreeze, and price update. Include the initiating account, the affected

account where applicable, the previous and new values, and the block timestamp. Ensure that calling `pause_contract` produces exactly one pause event.

Long term, maintain a documented and versioned schema for cNEAR-specific administrative events. Add integration tests that compare emitted events with the resulting state transitions, and configure operational monitoring to consume an event for every security-relevant administrative method.

12. Force transfers bypass the pause and freeze controls without documentation

Severity: **Informational**	Difficulty: **Not Applicable**
Type: Access Controls	Finding ID: TOB-CNEAR-12
Target: `src/lib.rs`, `README.md`	

Description

`force_ft_transfer` checks only the `Owner` role and one yoctoNEAR before calling `internal_transfer`, consulting neither `self.paused` nor `self.frozen_accounts`, so the owner can move tokens while transfers are paused and when either account is frozen. The documentation instead states that pausing stops all token transfers and that freezing prevents an account from transferring, so holders, integrators, and incident responders may assume that either control halts all movement. Authorization itself holds, so the consequence is that the documented guarantees overstate what the controls enforce.

```
- **Pausable** - Owner can pause/unpause all token transfers
- **Account freezing** - Owner can freeze individual accounts to prevent their
transfers
- **Force transfers** - Owner can move tokens between any accounts
```

Figure 12.1: The feature summary omits the force-transfer exemption. (README.md#L14–L16)

```
#[payable]
#[access_control_any(roles(Role::Owner))]
pub fn force_ft_transfer(
    &mut self,
    sender_id: AccountId,
    receiver_id: AccountId,
    amount: U128,
    memo: Option<String>,
) {
    assert_one_yocto();
    let amount: u128 = amount.into();
    self.token
        .internal_transfer(&sender_id, &receiver_id, amount, memo);
}
// ...
fn ft_transfer(&mut self, receiver_id: AccountId, amount: U128, memo:
Option<String>) {
    require!(!self.paused, "Token transfers paused");
    let sender_id = env::predecessor_account_id();
    require!(
        !self.frozen_accounts.contains(&sender_id),
```

```
            "Sender account is frozen"
        );
        require!(
            !self.frozen_accounts.contains(&receiver_id),
            "Receiver account is frozen"
        );
        self.token.ft_transfer(receiver_id, amount, memo)
    }
```

Figure 12.2: Force transfers omit both checks the ordinary transfer path applies. (src/lib.rs#L176–L207)

Exploit Scenario

During an incident involving suspected theft, governance pauses the contract and freezes the affected accounts, and responders communicate that these actions prevent token movement. An authorized owner, or an attacker who has compromised the owner account, then calls `force_ft_transfer` to move tokens from a frozen account while the pause remains active, and the transaction succeeds contrary to the expectations the documentation creates.

Recommendations

Short term, document that pause and freeze restrict only the ordinary NEP-141 transfer paths and that the owner may call `force_ft_transfer` in any state, and consider requiring `sender_id` to be frozen already so that enforcement matches the documentation, accepting that the owner can still freeze and then transfer in two transactions. This prevents the issue because holders and integrators will know the true scope of each control, and a required freeze precedes every forced transfer with an observable state change.

Long term, define a security-control interaction matrix covering how each privileged operation behaves under pause, freeze, and every other emergency state, and enforce it with one test per operation and state. This prevents the issue because coverage will be decided in one reviewed place rather than drifting method by method as entrypoints are added.

13. Ownership transfer retains the previous owner's administrative permissions

Severity: **High**	Difficulty: **High**
Type: Access Controls	Finding ID: TOB-CNEAR-13
Target: `src/lib.rs`	

Description

`owner_set` revokes only the previous owner's `Owner` role grant, but leaves intact the super-admin and `Owner`-role administrator permissions that `grant_owner_role` assigned at initialization. Because `near-plugins` 0.5.1 authorizes any super-admin to call `acl_grant_role`, `acl_revoke_role`, `acl_revoke_admin`, and `acl_revoke_super_admin` against any account, a former owner can either grant itself the `Owner` role back, or revoke all three permissions from the new owner. The second path leaves the new owner permanently unable to act or self-recover, and the former owner alone in control, while `owner_get` continues to name the new owner.

```
#[access_control_any(roles(Role::Owner))]
pub fn owner_set(&mut self, new_owner: AccountId) {
    let old_owner = self.owner_id.clone();
    self.owner_id = new_owner.clone();
    // Transfer role to new owner
    self.acl_revoke_role(Role::Owner.into(), old_owner);
    self.grant_owner_role(&new_owner);
}

fn grant_owner_role(&mut self, owner: &AccountId) {
    let mut acl = self.acl_get_or_init();
    acl.add_super_admin_unchecked(owner);
    acl.add_admin_unchecked(Role::Owner, owner);
    acl.grant_role_unchecked(Role::Owner, owner);
}
```

Figure 13.1: Ownership transfer revokes the role but retains the previous owner's administrative permissions. (`src/lib.rs#L160-L174`)

Exploit Scenario

An owner transfers ownership to a replacement account after suspecting its key has been exposed, but the former owner's key is still able to call `acl_grant_role` and restore its own `Owner` role, then call `upgrade` with unauthorized contract code or `force_ft_transfer` to move token balances. Alternatively, the former owner's key can instead call `acl_revoke_role`, `acl_revoke_admin`, and `acl_revoke_super_admin`

against the new owner, removing every permission the new owner holds so it can neither act nor self-recover, then regrant itself `Owner` and hold sole control while `owner_get` still names the new owner.

Recommendations

Short term, update `owner_set` to grant the required role and administrative permissions to the new owner and then revoke the previous owner's `Owner` role, `Owner`-role administrator permission, and super-admin permission, checking the result of each access-control operation and aborting the transfer if any required update fails. This prevents the issue because the previous owner retains no permission that allows it to restore its own `Owner` role or to revoke the new owner's.

Long term, implement the long-term recommendation for TOB-CNEAR-9.

14. Frozen or paused accounts can burn their balance via storage_unregister, defeating owner recovery

Severity: **Medium**	Difficulty: **Low**
Type: Access Controls	Finding ID: TOB-CNEAR-14
Target: `src/lib.rs`	

Description

The pause and freeze controls are enforced only in `ft_transfer` and `ft_transfer_call`, so a holder whose account is frozen, or any holder while the contract is paused, can still call `storage_unregister` with `force` set to true. That call routes through the standard `internal_storage_unregister`, which deletes the caller's registration, subtracts the caller's entire balance from `total_supply`, and refunds only the storage minimum plus one yoctoNEAR, so the tokens are destroyed rather than redeemed for any proportional NEAR. Because the balance no longer exists afterward, the owner can no longer recover or seize it through `force_ft_transfer`, so pausing and freezing do not lock a holder's funds.

```
#[payable]
fn storage_unregister(&mut self, force: Option<bool>) -> bool {
    #[allow(unused_variables)]
    if let Some((account_id, balance)) =
self.token.internal_storage_unregister(force) {
        log!("Closed @{} with {}", account_id, balance);
        true
    } else {
        false
    }
}
```

Figure 14.1: The `storage_unregister` method delegates to `internal_storage_unregister` with no pause or freeze check. (`src/lib.rs#L274–L283`)

Exploit Scenario

An owner freezes an account, or pauses the contract, during a compliance hold or a suspected-theft incident, intending to preserve the balance for later seizure. Before the owner calls `force_ft_transfer` to move the tokens, the holder calls `storage_unregister` with `force` set to true and one yoctoNEAR attached. The call burns the holder's entire balance, reduces `total_supply`, and deletes the registration, and the owner's subsequent `force_ft_transfer` fails because the account and its balance no longer exist.

Recommendations

Short term, reject `storage_unregister` when the contract is paused or the caller is frozen, or document that force-unregister is an intentional escape hatch that both controls permit. This prevents the issue because the value-destroying path will enforce the same pause and freeze policy as the transfer methods.

Long term, define a single written policy stating which operations pause and freeze block, and enforce it at one shared checkpoint that every value-moving and state-changing path passes through, with a test per path. This prevents the issue because enforcement will not drift method by method as new entrypoints are added.

A. Vulnerability Categories

The following tables describe the vulnerability categories, severity levels, and difficulty levels used in this document.

Vulnerability Categories	
Category	**Description**
Access Controls	Insufficient authorization or assessment of rights
Auditing and Logging	Insufficient auditing of actions or logging of problems
Authentication	Improper identification of users
Configuration	Misconfigured servers, devices, or software components
Cryptography	A breach of system confidentiality or integrity
Data Exposure	Exposure of sensitive information
Data Validation	Improper reliance on the structure or values of data
Denial of Service	A system failure with an availability impact
Error Reporting	Insecure or insufficient reporting of error conditions
Patching	Use of an outdated software package or library
Session Management	Improper identification of authenticated users
Testing	Insufficient test methodology or test coverage
Timing	Race conditions or other order-of-operations flaws
Undefined Behavior	Undefined behavior triggered within the system

Severity Levels	
Severity	**Description**
Informational	The issue does not pose an immediate risk but is relevant to security best practices.
Undetermined	The extent of the risk was not determined during this engagement.
Low	The risk is small or is not one the client has indicated is important.
Medium	User information is at risk; exploitation could pose reputational, legal, or moderate financial risks.
High	The flaw could affect numerous users and have serious reputational, legal, or financial implications.

Difficulty Levels	
Difficulty	**Description**
Not Applicable	This issue is of informational severity and does not pose an immediate risk, so difficulty does not apply.
Undetermined	The difficulty of exploitation was not determined during this engagement.
Low	The flaw is well known; public tools for its exploitation exist or can be scripted.
Medium	An attacker must write an exploit or will need in-depth knowledge of the system.
High	An attacker must have privileged access to the system, may need to know complex technical details, or must discover other weaknesses to exploit this issue.

B. Code Maturity Categories

The following tables describe the code maturity categories and rating criteria used in this document.

Code Maturity Categories	
Category	**Description**
Arithmetic	The proper use of mathematical operations and semantics
Auditing	The use of event auditing and logging to support monitoring
Authentication / Access Controls	The use of robust access controls to handle identification and authorization and to ensure safe interactions with the system
Complexity Management	The presence of clear structures designed to manage system complexity, including the separation of system logic into clearly defined functions
Cryptography and Key Management	The safe use of cryptographic primitives and functions, along with the presence of robust mechanisms for key generation and distribution
Decentralization	The presence of a decentralized governance structure for mitigating insider threats and managing risks posed by contract upgrades
Documentation	The presence of comprehensive and readable codebase documentation
Low-Level Manipulation	The justified use of inline assembly and low-level calls
Testing and Verification	The presence of robust testing procedures (e.g., unit tests, integration tests, and verification methods) and sufficient test coverage
Transaction Ordering	The system's resistance to transaction-ordering attacks

Rating Criteria	
Rating	**Description**
Strong	No issues were found, and the system exceeds industry standards.
Satisfactory	Minor issues were found, but the system is compliant with best practices.
Moderate	Some issues that may affect system safety were found.
Weak	Many issues that affect system safety were found.
Missing	A required component is missing, significantly affecting system safety.
Not Applicable	The category does not apply to this review.
Not Considered	The category was not considered in this review.
Further Investigation Required	Further investigation is required to reach a meaningful conclusion.

C. Test for TOB-CNEAR-5 and TOB-CNEAR-7

This appendix contains a test to reproduce the failure in TOB-CNEAR-5. The test adds the release metadata and blob, deliberately omits `add_deployment_info`, and confirms that `unrestricted_upgrade` fails with the controller's "hasn't been deployed" error. The test then adds the deployment information and verifies that the upgrade succeeds.

However, note that the successful upgrade does not mean that the contract is functional, as it still suffers from TOB-CNEAR-7.

```
use near_sdk::serde_json::json;
use near_workspaces::types::NearToken;
use std::path::PathBuf;

fn wasm(name: &str) -> Result<Vec<u8>, Box<dyn std::error::Error>> {
    let target = std::env::var("CARGO_TARGET_DIR").unwrap_or_else(|_|
"target".to_string());
    Ok(std::fs::read(
        PathBuf::from(target)
            .join("near")
            .join(format!("{name}.wasm")),
    )?)
}

#[tokio::test]
async fn registration_allows_controller_upgrade_but_target_is_invalid(
) -> Result<(), Box<dyn std::error::Error>> {
    let worker = near_workspaces::sandbox().await?;
    let dao = worker.dev_create_account().await?;
    let controller_wasm = wasm("aurora-controller-factory")?;
    let token_wasm = wasm("fungible_token")?;

    let controller_account = dao
        .create_subaccount("controller-repro")
        .initial_balance(NearToken::from_near(10))
        .transact()
        .await?
        .result;
    let controller = controller_account.deploy(&controller_wasm).await?.result;
    controller_account
        .call(controller.id(), "new")
        .args_json(json!({ "dao": dao.id() }))
        .transact()
        .await?
        .into_result()?;

    let token_account = dao
        .create_subaccount("token-repro")
        .initial_balance(NearToken::from_near(10))
        .transact()
```

```
        .await?
        .result;
    let token = token_account.deploy(&token_wasm).await?.result;
    dao.call(token.id(), "new")
        .args_json(json!({
            "owner_id": dao.id(),
            "total_supply": "1000000000000000",
            "metadata": {
                "spec": "ft-1.0.0",
                "name": "Controlled NEAR",
                "symbol": "cNEAR",
                "decimals": 24
            },
            "latest_price": "100000000000000000000000"
        }))
        .transact()
        .await?
        .into_result()?;
    dao.call(token.id(), "owner_set")
        .args_json(json!({ "new_owner": controller.id() }))
        .transact()
        .await?
        .into_result()?;

    let hash = near_sdk::env::sha256(&token_wasm)
        .iter()
        .map(|byte| format!("{byte:02x}"))
        .collect::<String>();
    dao.call(controller.id(), "add_release_info")
        .deposit(NearToken::from_yoctonear(1))
        .args_json(json!({
            "hash": hash,
            "version": "1.0.0",
            "is_latest": true,
            "downgrade_hash": null,
            "description": "cNEAR"
        }))
        .transact()
        .await?
        .into_result()?;
    dao.call(controller.id(), "add_release_blob")
        .deposit(NearToken::from_yoctonear(1))
        .args(token_wasm)
        .max_gas()
        .transact()
        .await?
        .into_result()?;

    // Match the deployment script and README: omit add_deployment_info.
    let result = dao
        .call(controller.id(), "unrestricted_upgrade")
        .deposit(NearToken::from_yoctonear(1))
        .args_json(json!({
```

```
            "contract_id": token.id(),
            "hash": hash,
            "state_migration_gas": null
        }))
        .max_gas()
        .transact()
        .await?;

    assert!(
        result.is_failure(),
        "an unregistered token must not upgrade"
    );
    let failure = format!("{result:#?}");
    assert!(
        failure.contains("hasn't been deployed"),
        "unexpected failure: {}",
        failure
    );

    dao.call(controller.id(), "add_deployment_info")
        .deposit(NearToken::from_yoctonear(1))
        .args_json(json!({
            "contract_id": token.id(),
            "deployment_info": {
                "hash": hash,
                "version": "1.0.0",
                "deployment_time": 0u64,
                "upgrade_times": {},
                "init_args": ""
            }
        }))
        .transact()
        .await?
        .into_result()?;

    let registered_upgrade = dao
        .call(controller.id(), "unrestricted_upgrade")
        .deposit(NearToken::from_yoctonear(1))
        .args_json(json!({
            "contract_id": token.id(),
            "hash": hash,
            "state_migration_gas": null
        }))
        .max_gas()
        .transact()
        .await?;
    assert!(
        registered_upgrade.is_success(),
        "registration should allow the controller upgrade transaction: {:#?}",
        registered_upgrade
    );

    let post_upgrade_call = dao.call(token.id(), "owner_get").view().await;
```

```
    assert!(
        post_upgrade_call.is_err(),
        "controller success should not be mistaken for a healthy upgraded cNEAR"
    );
    let post_upgrade_error = format!("{:#?}", post_upgrade_call.unwrap_err());
    assert!(
        post_upgrade_error.contains("CompilationError"),
        "unexpected post-upgrade failure: {}",
        post_upgrade_error
    );
    Ok(())
}
```

Figure C.1: Test to reproduce the failure in TOB-CNEAR-5 and TOB-CNEAR-7

D. Fix Review Results

When undertaking a fix review, Trail of Bits reviews the fixes implemented for issues identified in the original report. This work involves a review of specific areas of the source code and system configuration, not comprehensive analysis of the system.

On August 11, 2026, Trail of Bits reviewed the fixes and mitigations implemented by the cAssets Management team for the issues identified in this report. We reviewed each fix to determine its effectiveness in resolving the associated issue.

In summary, of the 14 issues described in this report, cAssets Management has resolved all of them. For additional information, please see the Detailed Fix Review Results below.

At the time of the review, all of the referenced commits were part of PR #10, whose top-most commit is currently `4d05e8c`. If the PR were to not be merged, or if it were force-pushed to a different commit, the fixes would have to be reevaluated.

The fixes include two notable changes that we would like to highlight. The first is that the contracts are now deployed using a Rust binary (`deploy-cli`) rather than a Bash script (commit `6a7347c`). We consider this a positive change, and we have convinced ourselves that the findings that applied to the Bash script (TOB-CNEAR-3 through TOB-CNEAR-6) do not apply to the Rust binary. Still, we have not audited the Rust binary in its entirety.

The second notable change was to the `justfile`. Per our recommendation (TOB-CNEAR-8), the Aurora controller commit is now pinned. However, that commit (`10f6729`) is one commit ahead of what was current at the time of the audit (`351dc02`).

In light of these two changes (use of an unaudited deployment binary and use of new supporting code), we recommend that cAssets Management seek additional review prior to deployment.

ID	Title	Severity	Status
1	Locked dependencies contain known security vulnerabilities	Informational	Resolved
2	Deployment retains access keys that bypass contract-governed administration	Informational	Resolved
3	Default deployment creates only one-billionth of one cNEAR	Informational	Resolved

4	Account-existence checks treat RPC errors as existing accounts	Low	Resolved
5	Deployment omits controller registration required for cNEAR upgrades	Low	Resolved
6	Ownership verification failures do not fail deployment	Low	Resolved
7	Controller-mediated upgrades deploy malformed contract code	High	Resolved
8	Unpinned controller source can change privileged deployment code	Informational	Resolved
9	owner_get can disagree with effective ownership	Informational	Resolved
10	Freeze-list growth uses contract-funded storage contrary to documentation	Informational	Resolved
11	Pause, freeze, and price changes do not emit dedicated events	Low	Resolved
12	Force transfers bypass the pause and freeze controls without documentation	Informational	Resolved
13	Ownership transfer retains the previous owner's administrative permissions	High	Resolved
14	Frozen or paused accounts can burn their balance via storage_unregister, defeating owner recovery	Medium	Resolved

Detailed Fix Review Results

TOB-CNEAR-1: Locked dependencies contain known security vulnerabilities
Resolved in commit `ae54ca7`. The lockfile was updated. Running `cargo audit` no longer produces errors.

TOB-CNEAR-2: Deployment retains access keys that bypass contract-governed administration
Resolved in commit `546f670`. The finding was resolved through documentation. The project's README now gives instructions for listing a contract's keys, removing its keys, and verifying that the keys have been removed.

TOB-CNEAR-3: Default deployment creates only one-billionth of one cNEAR
Resolved in commit `5a28e9c`. This change follows the switch to using a Rust binary for deployment. In commit `5a28e9c`, the default supply is defined as 10^14 whole cNEAR. The `tokens_to_yocto` function converts this value to atomic units by multiplying it by 10^decimals, which is 10^24 with the default configuration. Thus, the default supply passed to contract initialization is 10^38 atomic units, rather than the previous 10^15.

TOB-CNEAR-4: Account-existence checks treat RPC errors as existing accounts
Resolved in commit `6a7347c`. This commit adds the `deploy-cli` Rust binary. The binary does not use `grep`, and therefore does not have the overly broad `grep -q "Account"` behavior that the Bash script had.

TOB-CNEAR-5: Deployment omits controller registration required for cNEAR upgrades
Resolved in commit `7f00380`. The Rust deployment workflow now registers cNEAR with the controller before reporting deployment success. It computes the deployed token Wasm's SHA-256 hash, registers the initial release with `add_release_info`, uploads the corresponding code with `add_release_blob`, and creates the required deployment record using `add_deployment_info` (figure D.1). It then queries `get_deployment` and verifies that the recorded hash matches the deployed Wasm. Consequently, subsequent controller-mediated upgrades can find the required deployment record instead of failing with the "hasn't been deployed" error, as they did before.

```
signer
    .call(controller.id(), "add_deployment_info")
    .args_json(json!({
        "contract_id": token.id(),
        "deployment_info": {
            "hash": &hash,
            "version": RELEASE_VERSION,
            "deployment_time": 0u64,
            "upgrade_times": {},
            "init_args": "",
```

```
        },
    }))
    .deposit(NearToken::from_yoctonear(1))
    .max_gas()
    .transact()
    .await?
    .into_result()
    .context("registering the deployment with the controller failed")?;
```

Figure D.1: Call to `add_deployment_info` added by commit `7f00380` (cNEAR/deploy-cli/src/lib.rs#L911–L928)

TOB-CNEAR-6: Ownership verification failures do not fail deployment
Resolved in commit `6a7347c` (the same commit as TOB-CNEAR-4). The Bash ownership check has been replaced with a structured Rust view call (figure D.2). If the call or the check fails, the error is propagated to the caller, thereby resolving the issue.

```
let owner: String = signer.call(token.id(), "owner_get").view().await?.json()?;
if owner != controller.id().as_str() {
    bail!(
        "ownership verification failed: expected {}, got {}",
        controller.id(),
        owner
    );
}
```

Figure D.2: Ownership check in `deploy-cli` (cNEAR/deploy-cli/src/lib.rs#L804–L811)

TOB-CNEAR-7: Controller-mediated upgrades deploy malformed contract code
Resolved in commit `ec95288`. The upgrade entrypoint now declares the same Borsh-serialized parameters used by the Aurora controller. Moreover, the README now provides a sample command the operator can use to check whether the upgrade was successful.

TOB-CNEAR-8: Unpinned controller source can change privileged deployment code
Resolved commit `7e76790` and commit `1dabc91`. The `justfile` was updated to pin the Aurora controller to commit `0f67290`. Moreover, the `justfile` now verifies that the repository is clean before attempting to build the controller Wasm (figure D.3).

```
git -C "$DIR" checkout --quiet --detach {{controller-commit}}
# Remove untracked and ignored files (stray .cargo/config.toml, previous build
# artifacts, etc.) so the build input is exactly the pinned tree.
git -C "$DIR" clean -ffdxq
```

Figure D.3: Checks the `justfile` performs before attempting to build the Aurora controller Wasm (cNEAR/justfile#L22–L25)

One thing the revised `justfile` does not do is verify the hash of the Wasm that is built. We asked cAssets Management whether this was feasible, and got the following response:

> *On the determisitic wasm, we didn't prioritise it because this a contract with complete admin control so you have to trust the deployer anyway. The pipeline for deterministic deploys on NEAR has historically been very troublesome (failing/not actually being deterministic), so we didn't think the juice was worth the squeeze.*

TOB-CNEAR-9: owner_get can disagree with effective ownership
Resolved in commit `9f27d5a`. The generic role-based access-control mechanism was removed. The `owner_id` is now the single source of truth for both ownership reporting and authorization. All privileged methods are now protected by `#[only(owner)]`, whose authorization check compares the caller directly against the same `owner_id` returned by `owner_get`.

Additionally, ownership transfers now use a two-step propose-and-accept process. Completing a transfer replaces the singular `owner_id`, ensuring that the previous owner retains no privileged access.

TOB-CNEAR-10: Freeze-list growth uses contract-funded storage contrary to documentation
Resolved in commit `d87594f`. The documentation mismatch is resolved. The file-level comment now explicitly states the following:

- `freeze_account` is nonpayable.
- Freeze-list storage is funded from the contract account's balance.
- Removing an entry through `unfreeze_account` makes the associated storage stake available to the contract again but does not refund the caller.
- Further freezes can fail if the contract lacks sufficient available balance.

While we consider the finding resolved, we note that cNEAR provides no method for withdrawing general-purpose NEAR from the contract account. If an operator transfers additional NEAR to increase freeze-list capacity, those funds can support current or future storage usage but cannot subsequently be recovered through the contract interface. We recommend exploring the addition of a "withdraw" method.

TOB-CNEAR-11: Pause, freeze, and price changes do not emit dedicated events
Resolved in commit `2c7f76d`. The contract now defines versioned NEP-297 events for all affected administrative operations, including ownership changes and privileged forced transfers:

- `Paused`
- `Unpaused`
- `AccountFrozen`
- `AccountUnfrozen`
- `PriceUpdated`
- `OwnerChanged`
- `ForceFtTransfer`

Each event contains information pertinent to the operation it describes, such as the initiating account, affected account, transferred amount, or previous and new values. The methods that emit these events do so after performing their associated state changes, with no subsequent fallible contract logic.

TOB-CNEAR-12: Force transfers bypass the pause and freeze controls without documentation
Resolved in commit `5397d78`. The implementation intentionally continues to allow `force_ft_transfer` while the contract is paused or either account is frozen. The README now documents this behavior explicitly, making the following points:

- Pause and freeze restrict only non-owner methods.
- Owner methods such as force transfer are never blocked.
- Force transfers work regardless of pause or freeze state.

Additionally, doc comments were added to `pause`, `freeze_account`, and `force_ft_transfer`. The doc comments reinforce the points above.

TOB-CNEAR-13: Ownership transfer retains the previous owner's administrative permissions
Resolved in commit `9f27d5a` (the same commit as TOB-CNEAR-9). As explained in the fix description for TOB-CNEAR-9, the cNEAR contract no longer uses a role-based access-control mechanism, and instead uses the `owner_id` field as the single source of truth. Since this field is rewritten when a new owner accepts ownership, there is no way for the previous owner to retain their privileges.

TOB-CNEAR-14: Frozen or paused accounts can burn their balance via storage_unregister, defeating owner recovery
Resolved in commit `da5f8f5`. `storage_unregister` now rejects every attempt to unregister an account with a positive cNEAR balance (figure D.4).

```
require!(
    self.token.ft_balance_of(env::predecessor_account_id()).0 == 0,
    "cannot unregister an account that still holds cNEAR: transfer the balance out
first"
);
```

Figure D.4: Check that was added to cNEAR's `storage_unregister` method (cNEAR/src/lib.rs#L396–L399)

This is stronger and simpler than checking only whether the contract is paused or the caller is frozen. The changed semantics are documented in the README. Unit tests cover frozen and paused accounts, and an integration test confirms that `force: true` fails without reducing total supply.

E. Fix Review Status Categories

The following table describes the statuses used to indicate whether an issue has been sufficiently addressed.

Fix Status	
Status	**Description**
Undetermined	The status of the issue was not determined during this engagement.
Unresolved	The issue persists and has not been resolved.
Partially Resolved	The issue persists but has been partially resolved.
Resolved	The issue has been sufficiently resolved.

About Trail of Bits

Founded in 2012 and headquartered in New York, Trail of Bits provides technical security assessment and advisory services to some of the world's most targeted organizations. We combine high-end security research with a real-world attacker mentality to reduce risk and fortify code. With 100+ employees around the globe, we've helped secure critical software elements that support billions of end users, including Kubernetes and the Linux kernel.

We maintain an exhaustive list of publications at https://github.com/trailofbits/publications, with links to papers, presentations, public audit reports, and podcast appearances.

In recent years, Trail of Bits consultants have showcased cutting-edge research through presentations at CanSecWest, HCSS, Devcon, Empire Hacking, GrrCon, LangSec, NorthSec, the O'Reilly Security Conference, PyCon, REcon, Security BSides, and SummerCon.

We specialize in software testing and code review assessments, supporting client organizations in the technology, defense, blockchain, and finance industries, as well as government entities. Notable clients include HashiCorp, Google, Microsoft, Western Digital, Uniswap, Solana, Ethereum Foundation, Linux Foundation, and Zoom.

To keep up with our latest news and announcements, please follow @trailofbits on X or LinkedIn and explore our public repositories at https://github.com/trailofbits. To engage us directly, visit our "Contact" page at https://www.trailofbits.com/contact or email us at info@trailofbits.com.

Trail of Bits, Inc.
228 Park Ave S #80688
New York, NY 10003
https://www.trailofbits.com
info@trailofbits.com

Notices and Remarks

Copyright and Distribution

Test Coverage Disclaimer

Trail of Bits performed all activities associated with this project in accordance with a statement of work and an agreed-upon project plan.

Security assessment projects are time-boxed and often rely on information provided by a client, its affiliates, or its partners. As a result, the findings documented in this report should not be considered a comprehensive list of security issues, flaws, or defects in the target system or codebase.

Trail of Bits uses automated testing techniques to rapidly test software controls and security properties. These techniques augment our manual security review work, but each has its limitations. For example, a tool may not generate a random edge case that violates a property or may not fully complete its analysis during the allotted time. A project's time and resource constraints also limit their use.